

PRODUCT REQUIREMENTS DOCUMENT

Personal Finance OS

Intelligent Bank Statement Ingestion, Categorization & AI-Powered Insights

Version	1.1 — Improved PRD
Status	Draft — Ready for Review
Target Region	India (INR, Indian banks, UPI)
MVP Horizon	3–6 months (7 sprints)
Tech Lead Review	Required before Sprint 1

1. Product Overview

The Personal Finance OS is a Supabase-powered application that transforms raw bank statements (PDFs, CSVs, Excel) into structured, searchable, and AI-enriched financial intelligence. Rather than a simple calculator, it functions as a self-learning financial brain: it ingests raw transaction data, normalizes merchant names, auto-categorizes expenses, groups them into user-defined events (e.g. "Goa Trip 2026"), and answers natural-language queries against real SQL data.

Core Value Proposition

Unlike generic expense apps, this system learns from corrections, supports multiple Indian bank accounts and UPI, and prevents hallucinated answers by grounding every AI response in verified database queries — not LLM guesses.

1.1 Problem Statement

Indian tech-savvy professionals and freelancers manage 3–5 financial accounts simultaneously (savings, salary, credit card, UPI). Pain points include:

- Manual categorization: spending 30–60 minutes per month tagging transactions in spreadsheets.
- No event-level visibility: no easy way to know "how much did the Goa trip cost in total?"
- Fragmented data: different banks produce incompatible statement formats.
- Zero learning: corrections made today must be repeated next month.

1.2 Proposed Solution

A pipeline-driven system with five core behaviors:

1. Ingest — upload any bank PDF/CSV/Excel and extract structured transactions automatically.
2. Deduplicate — fingerprint every transaction so re-uploading overlapping statements is safe.
3. Normalize — resolve messy merchant strings ("Hdfc Bank Atm 1234") to canonical names ("HDFC ATM").
4. Categorize — layer rule-based, regex-based, and LLM-assisted categorization with a confidence score; remember user corrections permanently.
5. Converse — answer natural-language queries ("Why did I spend more in May?") using SQL-backed RAG, never letting the LLM compute totals directly.

2. Goals & Success Metrics

2.1 Business Goals

- Eliminate manual expense tracking for target users entirely within 30 days of onboarding.

- Achieve >90% auto-categorization accuracy, improving month-over-month via user feedback.
- Build sufficient user trust (NPS > 80) to support word-of-mouth growth in the Indian tech community.
- Maintain a cost structure sustainable below \$400/mo at 100K MAU.

2.2 Success Metrics

Metric	Target	Measurement Method
Transaction Ingestion Accuracy	≥ 98%	Automated test suite against known bank statements
Duplicate Ingestion Rate	< 5%	Re-upload same statement, count extra rows inserted
Auto-Category Accuracy (Month 1)	≥ 80%	Manual audit of 200 random transactions/month
Auto-Category Accuracy (Month 6)	≥ 90%	Improving via accumulated correction feedback
Chat Query Correctness	≥ 95%	Pre-seeded test dataset; compare LLM answer vs SQL ground truth
P95 Query Response Time	< 5 seconds	Load testing with simulated 1K concurrent users
User NPS	> 80	In-app survey at Day 30 and Day 90
Monthly Active User Retention	> 60% at Day 30	Cohort retention analysis in analytics dashboard

2.3 Constraints

- Currency: Indian Rupee (INR) exclusively at MVP; multi-currency is post-MVP.
- Banks: target HDFC, ICICI, SBI, Axis, Kotak, IDFC First — with extensible parser architecture for others.
- Compliance: transaction data retained 5–7 years per AML regulations; PII never sent to LLMs.
- Budget: LLM costs must remain proportional to usage; OpenRouter pay-as-you-go with key rotation.

3. Target Users & User Stories

3.1 Primary Persona — "The Multi-Account Professional"

Tech-savvy individuals (software engineers, designers, product managers) aged 24–38 managing salary + savings + UPI + credit card accounts. They are comfortable with technology but unwilling to spend hours in spreadsheets. They want automated clarity, not manual entry.

Example User

Rohan, 31, senior software engineer in Pune. He has an HDFC salary account, an ICICI credit card, a Zerodha trading account, and uses GPay for UPI. He spends ~45 min/month reconciling statements and frequently forgets to categorize fuel or grocery transactions. He wants to know "how much did I actually spend last month" without opening four tabs.

3.2 Secondary Persona — "The Freelance Consultant"

Independent consultants and small business owners who mix personal and client expenses. They need to separate reimbursable costs, track project-related spend, and generate expense reports — all without hiring an accountant.

3.3 User Stories (Priority Order)

US-01 (Must Have)	As a user, I upload my HDFC bank PDF and see auto-categorized transactions within 60 seconds.
US-02 (Must Have)	As a user, I correct a miscategorized merchant once and it is remembered forever — no re-correction next month.
US-03 (Must Have)	As a user, I ask 'How much did I spend on food last month?' and receive an answer based on actual SQL results, not an AI guess.
US-04 (Must Have)	As a user, I upload the same statement twice and no duplicate transactions appear.
US-05 (Should Have)	As a user, I select 15 transactions from my Goa trip, name it 'Goa Trip 2026', and get an AI summary showing flights, hotel, meals, and total spend.
US-06 (Should Have)	As a user, I view a monthly dashboard showing top categories and year-on-year comparison.
US-07 (Should Have)	As a user, I ask 'Why did I spend more in May?' and the chat identifies the specific categories that increased.
US-08 (Nice to Have)	As a user, I set a monthly budget for 'Dining Out' and receive an alert at 80% utilization.
US-09 (Nice to Have)	As a user, I export a PDF expense report for a selected date range and event.

4. Core Features

4.1 Data Ingestion & Parsing

Support uploading PDFs, CSVs, and Excel files from Indian banks. Use an asynchronous Redis/Celery queue so uploads return immediately while parsing continues in background.

- PDF parsing: pdfplumber for digital (text-layer) PDFs; OCRmyPDF + Tesseract for scanned statements.
- CSV/Excel: pandas-based parser with configurable column mapping per bank template.
- Manual entry: REST endpoint for single transactions (useful for cash or non-bank expenses).
- Parser output: normalized JSON — { date, amount, raw_merchant, memo, account_id }.
- File tracking: SHA256 of file + page content stored in processing_jobs to avoid re-parsing.

Design Note
Build a bank-specific parser registry. Each bank (HDFC, ICICI, etc.) has its own column order, date format, and header rows. New banks are added by registering a new parser class — no changes to core pipeline.

4.2 Deduplication

Compute a deterministic fingerprint: SHA256(date + amount + raw_merchant). Before inserting, check for an existing row with the same fingerprint. If found, skip.

- Edge case: re-uploaded files with minor formatting differences (trailing spaces, encoding) are normalized before hashing.
- UI: a Duplicates Review screen shows any transactions that were skipped, with the reason.
- Manual override: users can force-insert a skipped transaction if they believe it is genuinely new.

4.3 Merchant Normalization

Convert messy raw strings to canonical merchant names. Three-layer approach:

6. Lookup table: exact and prefix-match against merchant_aliases (e.g. "SBUX" → "Starbucks").
 7. Regex rules: pattern-based matching (e.g. /HPCL|HP PETROL|HP PUMP/ → "HP Petrol").
 8. LLM agent: for strings that match neither, prompt the model with the raw string and recent context; store confirmed result back into merchant_aliases.
- Users can manually merge two merchants or confirm/reject an LLM suggestion.
 - All confirmations update merchant_aliases so future statements auto-resolve correctly.

4.4 Categorization Engine

Multi-layer categorization with explicit confidence scoring:

Layer 1 — Hard Rules	Merchant-to-category map (e.g. Uber → Transport, Swiggy → Food). Confidence: 1.0.
Layer 2 — Regex	Pattern match on raw_merchant or memo (e.g. /GROCERY KIRANA/ → Food). Confidence: 0.85.

Layer 3 — LLM Fallback	For confidence < 0.7, invoke LLM with merchant, amount, and surrounding transactions. Store LLM suggestion + confidence.
User Feedback Loop	Corrections are stored as learning_records. On next run, these override LLM for that merchant.
Review Queue	Transactions with confidence < 0.5 appear in a 'Needs Review' inbox for user action.

4.5 Event Detection & Grouping

Allow users to define named "events" — logical groupings of transactions (e.g. "Wedding Expenses", "Work Trip — Bangalore").

- UI: multi-select transactions → "Create Event" → enter name and optional metadata (dates, destination, participants).
- AI summarizer: on event creation, generate a breakdown table (category → subtotal → % of event total).
- Querying: events are first-class query targets — "How much did I spend on Goa Trip?" returns the event total.
- Future: periodic AI scan to suggest event candidates (e.g. cluster of hotel + flight + restaurant on same dates).

4.6 Chat / RAG Interface

A conversational interface that answers financial questions by combining SQL retrieval with LLM language generation.

Critical Design Constraint

The LLM never performs arithmetic. All sums, comparisons, and aggregations are computed by SQL queries. The LLM receives structured results (e.g. {food_total: 12400, travel_total: 8200}) and generates the natural-language explanation.

- Query types: totals by category, merchant breakdown, month-over-month comparison, event totals, top spending days.
- Fallback: if the query cannot be resolved by SQL, the system explains what it cannot answer rather than guessing.
- Context retrieval: pgvector search over embedded event summaries and user notes provides long-term memory without fitting all transactions in the LLM context window.

4.7 Multi-Model Gateway (OpenRouter)

- Primary: Google Gemini Flash (fast, cost-efficient for categorization).
- Secondary: Anthropic Claude (complex reasoning for ambiguous merchants and chat queries).
- Fallback: OpenRouter Nemotron Ultra (free, for when all paid keys are rate-limited).

- Key rotation: pool of keys cycled via OpenRouter Management API. Both old and new keys remain valid during rotation — zero downtime.
- Cost guard: only successful API calls are billed by OpenRouter. Failed/rate-limited attempts incur no charge.

4.8 Security & Privacy

- Supabase Auth handles all user management (email/password + OAuth options).
- Row-Level Security (RLS) enabled on every table: USING (auth.uid() = user_id). No cross-user data leakage possible at the DB layer.
- PII handling: full account numbers are truncated before storage; never sent to LLMs.
- TLS in transit, AES-256 at rest (Supabase default).
- Data deletion: users can request account + data deletion. System purges within 30 days, except anonymized aggregates (if user opts in to retention for their own trend data).

5. Data Model

5.1 Core Schema (Supabase / PostgreSQL)

```
-- Users (managed by Supabase Auth; this table extends it)
CREATE TABLE users (
  id          UUID PRIMARY KEY,
  email       TEXT UNIQUE NOT NULL,
  preferences JSONB DEFAULT '{}', -- e.g.
{"default_currency": "INR", "theme": "light"}
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Bank accounts (multi-account support)
CREATE TABLE accounts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  name        TEXT NOT NULL,          -- e.g. 'HDFC Salary Account'
  type        TEXT NOT NULL,          -- 'Bank' | 'CreditCard' | 'UPI' |
'Investment'
  bank_code   TEXT,                  -- e.g. 'HDFC', 'ICICI'
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Canonical merchants
CREATE TABLE merchants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT UNIQUE NOT NULL,   -- e.g. 'Starbucks'
  domain      TEXT,                  -- optional: 'starbucks.com'
  logo        TEXT                   -- optional: URL to logo asset
);

-- Merchant alias lookup (raw string → canonical merchant)
CREATE TABLE merchant_aliases (
  id          SERIAL PRIMARY KEY,
```

```
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    alias       TEXT NOT NULL,
    match_type  TEXT DEFAULT 'exact', -- 'exact' | 'prefix' | 'regex' |
'11m_confirmed'
    UNIQUE(alias)
);

-- Categories (hierarchical; system + user-custom)
CREATE TABLE categories (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE, -- NULL = system
    category    TEXT NOT NULL, -- e.g. 'Food', 'Transport', 'Groceries'
    parent_id   UUID REFERENCES categories(id),
    icon        TEXT           -- emoji or icon name for UI
);

-- Transactions (core table)
CREATE TABLE transactions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    account_id  UUID REFERENCES accounts(id),
    date        DATE NOT NULL,
    amount      NUMERIC(12,2) NOT NULL,
    currency    TEXT DEFAULT 'INR',
    raw_merchant TEXT, -- original string from statement
    merchant_id UUID REFERENCES merchants(id),
    category_id UUID REFERENCES categories(id),
    memo        TEXT,
    fingerprint TEXT UNIQUE NOT NULL, --
SHA256(date+amount+raw_merchant+account_id)
    is_transfer BOOLEAN DEFAULT FALSE, -- flagged transfers (salary-savings,
etc.)
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Category assignments with confidence (supports multiple tags per transaction)
CREATE TABLE transaction_categories (
    transaction_id UUID REFERENCES transactions(id) ON DELETE CASCADE,
    category_id    UUID REFERENCES categories(id),
    confidence     REAL NOT NULL DEFAULT 1.0, -- 0.0-1.0
    source         TEXT DEFAULT 'rule', -- 'rule' | 'regex' | '11m' | 'user'
    PRIMARY KEY (transaction_id, category_id)
);

-- User feedback / learning records
CREATE TABLE learning_records (
    id          SERIAL PRIMARY KEY,
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    raw_merchant TEXT NOT NULL,
    category_id UUID REFERENCES categories(id),
    merchant_id UUID REFERENCES merchants(id),
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(user_id, raw_merchant) -- one authoritative correction per merchant per
user
);

-- Events (user-defined transaction groups)
CREATE TABLE events (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    name        TEXT NOT NULL, -- e.g. 'Goa Trip 2026'
```



```

    metadata    JSONB,          -- {"destination":"Goa","days":4,"participants":2}
    summary     TEXT,           -- AI-generated summary text
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE event_transactions (
    event_id     UUID REFERENCES events(id) ON DELETE CASCADE,
    transaction_id UUID REFERENCES transactions(id) ON DELETE CASCADE,
    PRIMARY KEY (event_id, transaction_id)
);

-- Notes / summaries for RAG (embedded into pgvector)
CREATE TABLE notes (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID REFERENCES users(id) ON DELETE CASCADE,
    content      TEXT NOT NULL,
    source       TEXT DEFAULT 'system',      -- 'event_summary' | 'user_note' |
    'monthly_digest'
    created_at   TIMESTAMPTZ DEFAULT NOW(),
    embedding    vector(1536)               -- pgvector: text-embedding-3-small
);

-- File processing jobs (for dedup and retry tracking)
CREATE TABLE processing_jobs (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID REFERENCES users(id) ON DELETE CASCADE,
    file_hash    TEXT NOT NULL,
    status       TEXT DEFAULT 'queued',      -- 'queued' | 'processing' | 'done' | 'failed'
    error        TEXT,
    created_at   TIMESTAMPTZ DEFAULT NOW(),
    finished_at  TIMESTAMPTZ
);

```

5.2 RLS Policies

Every table has RLS enabled. Example:

```

ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users see own transactions"
    ON transactions FOR ALL
    USING (auth.uid() = user_id);

-- Repeat pattern for: accounts, events, notes, learning_records, processing_jobs

```

5.3 Vector Strategy

We do NOT embed every transaction (too many rows, too much cost). We embed only high-level summaries:

- Monthly digest: after month-end, generate and embed a ~200-word summary of spending by category.
- Event summaries: embed each event's AI-generated breakdown on creation.
- User notes: embed any manual note added by the user.

Chat queries do a cosine similarity search over the notes table to fetch the most relevant context before calling the LLM.

6. System Architecture

6.1 Component Overview

Frontend	Next.js 14 (App Router) + Tailwind CSS + TanStack Query. Handles file uploads, auth, transaction views, event creation, and chat UI.
Backend API	FastAPI (Python 3.12). Handles auth token validation, file upload ingestion, REST endpoints (/upload, /transactions, /events, /chat), and orchestrates pipeline invocations.
Pipeline / Agents	LangGraph directed graph: Parse → Dedupe → Merchant Normalize → Categorize → Store → Embed. Each node is a Python function; LangGraph manages state, retries, and conditional branches.
Queue	Redis + Celery (or Dramatiq). Parse/OCR is CPU-bound; queue ensures HTTP responses remain fast (<500ms) while workers process asynchronously.
Database	Supabase (PostgreSQL 15 + pgvector + Auth + Storage + RLS). Single-vendor for data, files, auth, and embeddings.
LLM Gateway	OpenRouter: multi-key pool, automatic fallback, BYOK. Keys rotated via OpenRouter Management API without provider key changes.
Observability	Prometheus metrics from Supabase scraping → Grafana dashboards. Sentry for API errors and performance. LangSmith for LLM trace debugging.

6.2 Pipeline Flow

End-to-end sequence for a statement upload:

9. User uploads file via Next.js UI.
10. FastAPI validates token, stores file in Supabase Storage, computes file hash.
11. If file hash already in processing_jobs → return early (already processed).
12. Enqueue parse job in Redis. Return job_id to client immediately.
13. Parser Worker: extract rows → normalize JSON. Output: list of raw transaction objects.
14. Dedupe Node: hash each row. Skip if fingerprint exists in transactions.
15. Merchant Normalize Node: lookup alias table → regex → LLM (if no match). Write to merchants + merchant_aliases.
16. Categorize Node: rules → regex → LLM (if low confidence). Write to transaction_categories.
17. Store: insert final transactions rows. Update processing_jobs status to 'done'.
18. Embed: for any new event or monthly digest, generate embedding and store in notes.
19. Frontend polls /job/:id status; on completion, refreshes transaction list.

6.3 Chat / RAG Flow

- 20. User types query into chat UI.
- 21. Backend NLU: classify intent (total_by_category, merchant_breakdown, period_comparison, event_query, open-ended).
- 22. SQL Retrieval: run parameterized query matching intent. Return structured result dict.
- 23. Vector Retrieval (if needed): cosine search notes table for relevant monthly digests or event summaries.
- 24. Prompt: pass SQL results + vector context to LLM. Instruct model to explain, not compute.
- 25. Stream response back to UI.

```
-- Example: total by category for May 2026
SELECT c.name AS category, COALESCE(SUM(t.amount), 0) AS total
FROM transactions t
JOIN categories c ON t.category_id = c.id
WHERE t.user_id = :uid
      AND t.date >= '2026-05-01'
      AND t.date < '2026-06-01'
      AND t.is_transfer = FALSE
GROUP BY c.id
ORDER BY total DESC;

-- LLM receives: {"Food": 12400, "Transport": 4200, "Entertainment": 3800, ...}
-- LLM explains: "In May, your largest spend was Food at ₹12,400 — 31% of
total..."
-- LLM never adds or compares numbers itself.
```

7. Technology Stack

Next.js 14 (App Router)	SSR + file upload support + Supabase JS client. Responsive dashboard and chat UI.
FastAPI (Python 3.12)	Async, type-safe, excellent Python AI library compatibility. /upload, /transactions, /events, /chat endpoints.
LangGraph	Directed pipeline graph with explicit state management. Conditional branches for retry/human-in-loop. Integrates with LangSmith for tracing.
Redis + Celery	Async task queue for parse jobs. Parse/OCR is CPU-bound and would block API threads.
Supabase (PostgreSQL 15)	Managed Postgres with Auth, RLS, Storage, and pgvector. Single vendor reduces DevOps complexity.
pgvector	Native Postgres vector extension. Avoids external Pinecone or Weaviate for MVP. Cosine similarity search over embeddings.
OpenRouter	Multi-model gateway. Key rotation, usage analytics, pay-per-successful-call billing. Nemotron fallback at no extra cost.
pdfplumber / PyMuPDF	Text-layer PDF extraction. Fast and accurate for digital bank statements.

OCRmyPDF + Tesseract	OCR fallback for scanned or image-based PDFs. Slower but covers edge cases.
Prometheus + Grafana	Supabase exposes ~200 DB metrics. Grafana for dashboards and alerting (slow queries, disk, connections).
Sentry	Error tracking and performance monitoring for FastAPI and frontend.
LangSmith	LangChain-native tracing for LangGraph pipelines. Debug mis-categorizations and prompt issues.

8. Alternatives Considered

Aspect	Supabase (Recommended)	MongoDB + Cloudinary	S3 + Postgres
Data Model	Relational SQL + JSONB. Native RLS per-row isolation.	Document model — flexible but analytics-heavy queries are awkward.	Relational, but no integrated auth/ACL; need manual stitching.
Analytics & Queries	SQL: SUM, GROUP BY, window functions. pgvector built-in for RAG.	Aggregation pipeline; complex financial queries are cumbersome.	Equally capable, but two services to manage and monitor.
File Storage	Built-in buckets sharing the same auth context.	Cloudinary: image/video CDN — overkill for PDF statements.	S3: reliable but separate IAM, credentials, and config.
Auth & Security	Integrated Auth + RLS policies. AES-256 encryption by default.	Requires Auth0/Clerk integration; row security must be manual.	AWS IAM or custom auth; Postgres RLS available but more config.
Vector Search	pgvector native — zero extra cost or service.	Atlas Vector Search or external Pinecone needed.	Can add pgvector; depends on managed Postgres provider.
Estimated Cost (50K MAU)	~\$100–200/mo (Supabase Pro, incl. backups & monitoring).	MongoDB Atlas + Cloudinary: separate billing, likely higher.	S3 cheap; managed Postgres comparable — but no all-in-one.
Dev Velocity	High — infra abstracted, team focuses on product features.	Moderate — full stack auth + file CDN setup needed.	Low-medium — many pieces to integrate.

Decision

Supabase is the clear choice: relational data model for financial queries, built-in RLS for security, pgvector for RAG, integrated auth, and competitive pricing up to 100K MAU. It eliminates the need to wire multiple vendors.

9. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
OCR / PDF Parsing Variability	High	High	Hybrid approach: pdfplumber for digital PDFs, Tesseract/OCRmyPDF for scans. Format-specific rules per bank. Always surface parsed results for user review.
LLM Category Hallucinations	Medium	High	LLM is advisory only (with confidence threshold). All financial calculations done via SQL — never by the LLM.
Duplicate Detection Failures	Low	Medium	Configurable dedupe tolerance. UI to manually merge detected duplicates. Fallback fingerprint fields (date + amount).
Merchant Alias Ambiguity	Medium	Medium	Combine alias with UPI remarks / merchant context. Periodic user-feedback prompts for ambiguous cases.
LLM Context Length Limits	High	Medium	Pre-aggregate monthly totals. Embed only summaries (not raw rows). RAG fetches targeted summaries for each chat query.
Transfer vs. Expense Confusion	Medium	Medium	Transfer detection rule: same-user account + within 24 hrs = 'Transfer' category. User can correct and the rule updates.
All API Keys Exhausted	Low	High	Multiple OpenRouter keys + auto-rotation. Graceful degradation: pause LLM tasks, queue for retry. Nemotron fallback always available.
Scaling to 100K+ Users	Medium	Medium	Index on (user_id, date). Partition old data. Materialized views for common aggregates. Upgrade to Supabase Team at 100K MAU.

10. MVP Roadmap (16 Weeks / 7 Sprints)

Sprint	Deliverable	Acceptance Criteria	Tests
S1 Wks 1–2	Supabase schema, RLS policies, Auth, FastAPI connectivity, seeded dummy data.	Auth flow works; cross-user RLS blocks access.	Auth unit tests; RLS policy tests with two test users.
S2 Wks 3–4	File upload endpoint + queue. PDF and CSV/Excel parsers. Transactions inserted into DB.	≥ 90% of lines from 5 test statements parsed correctly.	Unit tests per bank format; integration test end-to-end import.
S3 Wks 5–6	Deduplication logic (fingerprint). Merchant tables. Basic alias lookup.	Re-uploading same file produces zero new rows.	Overlapping-file test yields 0 duplicates; merchant alias resolves 80%+ of known aliases.
S4 Wks 7–8	Categorization engine: rules + regex + LLM	80% auto-tag coverage; user corrections persist	End-to-end: insert mock transactions, verify tags; correction persistence test.

Sprint	Deliverable	Acceptance Criteria	Tests
	fallback. Confidence scores. User correction UI.	and apply to future imports.	
S5 Wks 9–10	Event creation UI (multi-select). Event metadata. AI summarization of event spend.	User can group transactions, name event, receive logical AI breakdown.	Functional test: group 10 transactions, verify DB event rows and summary content.
S6 Wks 11–12	/chat endpoint. SQL-backed retrieval. LLM explanation layer. Streaming responses.	Chat answers match SQL ground truth within 2% variance on test dataset.	Automated: pre-seed data, run 20 known queries, compare against expected SQL results.
S7 Wks 13–16	Sentry + LangSmith integration. Grafana dashboards. Key rotation PoC. Security audit.	Zero uncaught exceptions in staging; key rotation runs without downtime.	Key rotation test with dummy keys; Sentry alert fires on injected error.

11. Operational Considerations

11.1 Backup & Disaster Recovery

- Supabase Pro: daily backups with 7-day retention. Point-in-time recovery available.
- Manual exports: scheduled weekly export of transactions table via API to long-term S3 archive.
- RTO target: < 4 hours. RPO target: < 24 hours (daily backup frequency).

11.2 Data Retention

- Default: transaction data retained for 7 years (Indian AML / tax audit compliance).
- User control: UI option to anonymize transactions older than a user-chosen date (keeps totals, drops PII).
- Right to erasure: full account deletion purges all rows within 30 days.

11.3 Cost Estimates

Supabase Pro (0–50K MAU)	~\$100–200/mo (base \$25 + storage + compute overages)
Supabase Team (50K–500K MAU)	~\$599/mo flat; covers 500K MAU
LLM costs (OpenRouter)	Pay-per-successful-call. ~100 chat queries/day at \$0.10 each ≈ \$300/mo at scale
Redis / Celery workers	Small VMs: negligible (~\$20/mo on any cloud provider)
Sentry	Free tier sufficient for MVP; Paid at \$26/mo for higher volume

Total at 100K MAU	Estimated \$600–900/mo all-in (infra + LLM); scales linearly with usage
-------------------	---

11.4 Key Rotation Strategy

- Maintain a pool of 3+ OpenRouter API keys in environment/secret manager.
- Rotation script runs nightly: create a new key via OpenRouter Management API, update pool, retire oldest key after 48-hour overlap window.
- On rate-limit error: immediately switch to next key in pool; alert on Sentry if pool size drops to 1.
- BYOK mode: rotating OpenRouter keys does not affect underlying Google/Anthropic provider credentials.

12. Testing Plan

Unit Tests (pytest)	Parser functions (feed sample PDFs/CSVs, assert JSON output). Dedupe logic. Merchant normalizer. Category assigner (mock LLM responses). RLS policy tests with two isolated users.
Integration Tests	End-to-end upload → parse → categorize → store pipeline. Multi-select event creation. Chat query → SQL retrieval → LLM answer pipeline.
Data Quality Checks	Monthly: sample 200 random transactions, manually audit category accuracy. Track error rate via user 'correct this' actions as a % of total transactions.
Load Tests (Locust)	Simulate 1K concurrent uploads and 500 concurrent chat queries. Target P95 < 5s. Run before each production deployment.
Security Tests	Attempt cross-user data access via manipulated JWT. File upload with malicious filenames and oversized payloads. LLM prompt injection tests (attempt to extract other users' data via chat).
User Acceptance	5 internal beta users with real bank statements. Collect NPS and bug reports. Accept if NPS > 70 and no P0 data-accuracy bugs.

13. Key API Endpoints

```
POST /api/upload-statement
  Body: multipart/form-data { file, account_id }
  Auth: Bearer JWT
  Response: { job_id, status: "queued" }

GET /api/jobs/{job_id}
  Response: { status: "queued"|"processing"|"done"|"failed", error? }

GET /api/transactions
  Query: ?start=2026-05-01&end=2026-05-31&category_id=&page=1&limit=50
```

```
Response: { data: Transaction[], total, page }

PATCH /api/transactions/{id}/category
Body: { category_id, merchant_correction?: boolean }
Response: { updated: true }

POST /api/events
Body: { name, transaction_ids: UUID[], metadata?: {} }
Response: { event_id, summary }

POST /api/chat
Body: { question: string, conversation_id?: string }
Response: Server-Sent Events stream of answer tokens
```

14. Glossary

RLS (Row-Level Security)	PostgreSQL feature restricting rows by per-user policy. Supabase enables this by default on all tables.
RAG (Retrieval-Augmented Generation)	Combining a database/vector retrieval step with LLM generation. Ensures answers are grounded in real data.
Deduplication	Removing duplicate transactions using a deterministic fingerprint to prevent double-counting on overlapping uploads.
Merchant Normalization	Converting raw merchant strings (e.g. 'SBUX 4432', 'Starbucks Sydney') to a canonical name ('Starbucks').
pgvector	PostgreSQL extension for storing and querying vector embeddings. Used here for chat memory and event retrieval.
LLM Hallucination	When an AI model fabricates details. Mitigated by never letting the LLM perform calculations — only DB results are used for numbers.
Fingerprint	SHA256(date + amount + raw_merchant + account_id). Uniquely identifies a transaction for deduplication purposes.
Learning Record	A stored user correction: maps a raw_merchant string to the user's preferred category. Applied on future imports.
Event	A user-defined logical grouping of transactions (e.g. 'Goa Trip 2026'). Supports AI summarization and direct querying.
OpenRouter	An LLM API gateway providing multi-model access, key management, and automatic fallback. Only bills successful calls.
BYOK (Bring Your Own Key)	Using your own provider API keys through OpenRouter, so rotating OpenRouter keys does not affect provider credentials.